

浙江大学

本科实验报告

课程名称: 电子电路系统综合实验

姓 名:

学 院: 信息与工程学院

专 业: 信息工程

学 号:

组 号:

项目名称: 便携式脉搏测试仪

指导教师: 金向东 王子立 龚淑君

2026 年 7 月 17 日

浙江大学实验报告

课程名称: _____ 电子电路系统综合实验 _____ 学生姓名: _____
实验名称: _____ 便携式脉搏测试仪 _____ 指导老师: _____ 金向东, 王子立, 龚淑君 _____

一、实验任务

设计并制作一个便携式人体脉搏测试仪, 该测试仪采用红光或红外光发射接收技术, 从人体手指或耳垂处采样获取脉搏信息, 并能实时显示被测者每分钟的脉搏数。其系统框图如图 1 所示, 其中 A、B 为 2 处信号观测点用于作品评测。

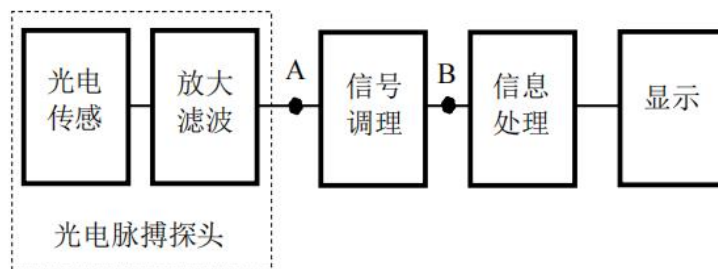


图 1 脉搏测试仪系统方框图

二、实验任务与要求

1、基本要求

- (1) 设计制作光电脉搏探头, 发射红外光或红光作为探测信号, 照射到指尖等人体组织后, 接收其透射或反射信号。
- (2) 设计制作脉搏信号调理电路与信息处理电路, 测量并显示被测人每分钟脉搏次数, 以医学仪器产品同时测量值为对照, 测量误差不大于 ± 3 次。
- (3) 测试仪采用 5V 单电源供电。
- (4) 测试仪在测量状态时, 能在光电探头达到合适测试部位时自动启动测量, 1 分钟完成测量后自动待机, 直至撤离探头并再次达到测试部位时自动启动下一次测量。

2. 发挥部分

- (1) 可预置脉搏次数上下告警门限, 当脉搏次数测量值超出告警限时, 测试仪告警。
- (2) 采集记录脉搏波形信号, ADC 的采样速率为 50Hz, 记录波形长度为 2 秒。并通过异步串口(UART, 采用 USB 转串口模块), 将波形信号数据采用“串口助手”传送给 PC(传输速率 115200bps), 并导出到 Excel 生成脉搏波形(也可采用第三方应用程序实时显示波形)。
- (3) 采用 MATLAB, C# 等设计应用程序, 接收串口数据, 显示波形。
- (4) 研究脉搏的快速检测方法, 在尽量短的时间内实现脉搏参数的检测(比如 20 秒内)。脉搏参数可通过串口传送 PC。
- (5) 实现通过蓝牙透传模块的无线传输, 通过“手机串口助手”APP 接收波形数据和心率(脉搏次数)数据。
- (6) 其它。

三、实验说明

- (1) 本题只能使用一个控制器。
- (2) 不允许使用光电发射接收器一体化成品。

四、实验仪器设备

计算机、STM32、LM358、光电传感器、OLED 显示屏、蓝牙模块、USB-TTL 转换器、蜂鸣器、按键开关、LED 灯、杜邦线、电阻、电容、面包板等。

五、实验原理

光电脉搏检测的基本原理是：随着心脏的搏动，人体组织半透明度随之改变。当血液送到人体组织时，组织的半透明度减小；当血液流回心脏，组织的半透明度增大，这种现象在人体组织较薄的手指指尖部位最为明显。

本设计利用波长 600-1000nm 的红光或红外发光二极管产生的光线照射到人体的手指尖、耳垂等部位，用装在该部位另一侧的光电接收管来检测机体组织的透明程度，即可将搏动信息转换成电信号。光电接收管输出的信号幅值极小（毫伏级），且叠加有环境光干扰、手指抖动噪声及高频噪声，因此需要经过放大、滤波和整形等一系列信号调理后，才能送入微控制器进行处理。

系统总体工作原理为：光电传感器将脉搏引起的光强变化转换为微弱电信号→两级放大滤波电路将信号放大并滤除噪声→比较整形电路将模拟脉搏波转换为数字脉冲→STM32 微控制器对脉冲计数并计算心率→OLED 显示结果，同时可通过串口/蓝牙将数据传输到 PC 或手机。

六、实验方案设计与参数计算

（一）各功能电路设计与计算

1、光电传感器接口电路

光电接收管将光强变化转换为电压信号。采用三脚排针和杜邦线与光电传感器相连，将输入电压信号引到电路板上。光电接收管输出的信号幅值通常在十几毫伏左右，需要后续放大电路进行处理。

2、低通滤波放大器电路

由于光电传感器输出的脉搏信号幅值较小，且容易受到环境光和手指抖动影响，所以本设计采用两级放大滤波结构。一级完成初步放大，二级进一步放大并滤除噪声，这样既能提高信号幅度，又能避免单级放大倍数过大导致运放饱和。

实验采用的运放芯片为 LM358，由于芯片本身限制，其输出电压最大值约为 3.8V，为了避免模拟波形出现双向削顶现象，因此小组调整了电阻和电容的取值，将电压放大到了合适的倍数。

光电传感器输出的信号先通过 C_1 隔直后送入一级运放放大，放大倍数由 R_4 和 R_5 决定：

$$A_{V1} = -\frac{R_5}{R_4} = -23.5$$

C_1 为隔直电容， C_2 与反馈电阻构成低通滤波，用于抑制高频噪声，保留脉搏信号的有效低频分量，截止频率为：

$$f_{L1} = \frac{1}{2\pi R_4 C_1} = 0.36\text{Hz}, f_{H1} = \frac{1}{2\pi R_5 C_2} = 33.86\text{Hz}$$

一级输出的脉搏信号继续进入二级放大滤波电路，放大倍数由 R_7 和 R_8 决定：

$$A_{V2} = -\frac{R_8}{R_7} = -5$$

经过二级放大，原本较为微弱的传感器信号被放大了约 118 倍，提高信号幅值的同时也通过低通滤波滤除手指抖动、环境干扰和高频噪声，使脉搏波形更加平滑稳定。此处运放 U1.2 的 7 脚输出为 A 端口输出，为模拟脉搏波形。

3、偏置电路

由于电路采用 5V 单电源供电，运放不能处理负电压信号，因此需要将信号抬升至正电压区间。偏置电压由 R_{15} 和 R_{16} 决定：

$$V_d = V_{CC} \times \frac{R_{16}}{R_{15} + R_{16}} = 1.69V$$

经过偏置电压处理与放大倍数选择，输出信号不再出现双向削顶现象，可以正常使用。

4、比较整形电路

放大滤波后的模拟脉搏波送入比较整形电路。电位器 V/R_1 用于调节比较阈值，将阈值电压设定在脉搏波信号的幅值范围内，U2.2 作为迟滞比较器，将模拟波形转换为数字脉冲信号。

R_1 和 R_{19} 用于分压，保证进入 STM32 的电压不超过 3.3V，保护单片机 I/O 口。此处运放 U2.2 的 7 脚输出为 B 端口输出，为整形后的数字脉冲。

5、蜂鸣器告警电路

STM32 通过 PB0 脚的 BEEP_CTRL 控制 S8050 三极管导通或截止，从而驱动 5V 有源蜂鸣器。当测得脉搏次数超过设定上下限时，蜂鸣器发出告警提示。S8050 为 NPN 三极管，基极串接限流电阻。

6、蓝牙与 USB 串口通信电路

蓝牙模块连接 PB10 和 PB11 与 STM32 通信，实现无线数据传输。USB-TTL 串口模块利用 STM32 开发板实现，接口使用 PA9 和 PA10，用于将脉搏波形数据和心率数据发送到 PC 串口助手。

7、OLED 显示电路

OLED 通过 I2C 接口与 STM32 连接，PB6 作为 SCL，PB7 作为 SDA。显示屏用于实时显示当前脉搏次数、测量状态以及告警信息。

（二）输入输出接口

本次实验 STM32 的输入输出接口如表 1 所示。

接口名称	引脚	功能说明	电平标准
模拟信号 A	PA1	ADC 采样（模拟脉搏波）	0~3.3V
数字脉冲 B	PB1	脉冲计数（整形脉冲）	0~3.3V
蜂鸣器控制	PB0	BEEP_CTRL	3.3V
蓝牙 TX	PB10	USART3_TX	3.3V
蓝牙 RX	PB11	USART3_RX	3.3V
USB TX	PA9	USART1_TX	3.3V
USB RX	PA10	USART1_RX	3.3V
OLED SCL	PB6	I2C_SCL	3.3V
OLED SDA	PB7	I2C_SDA	3.3V

表 1 STM32 输入输出接口

七、实验电路与程序设计

（一）硬件电路设计

1、电路图设计

实验完整的电路原理图包括光电传感器接口、两级放大滤波电路（U1.1 和 U1.2）、偏置电路（U2.1）、比较整形电路（U2.2）、STM32、OLED 显示模块、蜂鸣器告警电路、蓝牙模块接口、USB 串口接口、按键输入电路（SET/UP/DOWN）、LED 指示灯以及电源电路。所有电路均采用 5V 单电源供电。完整的电路图如图 2 所示。

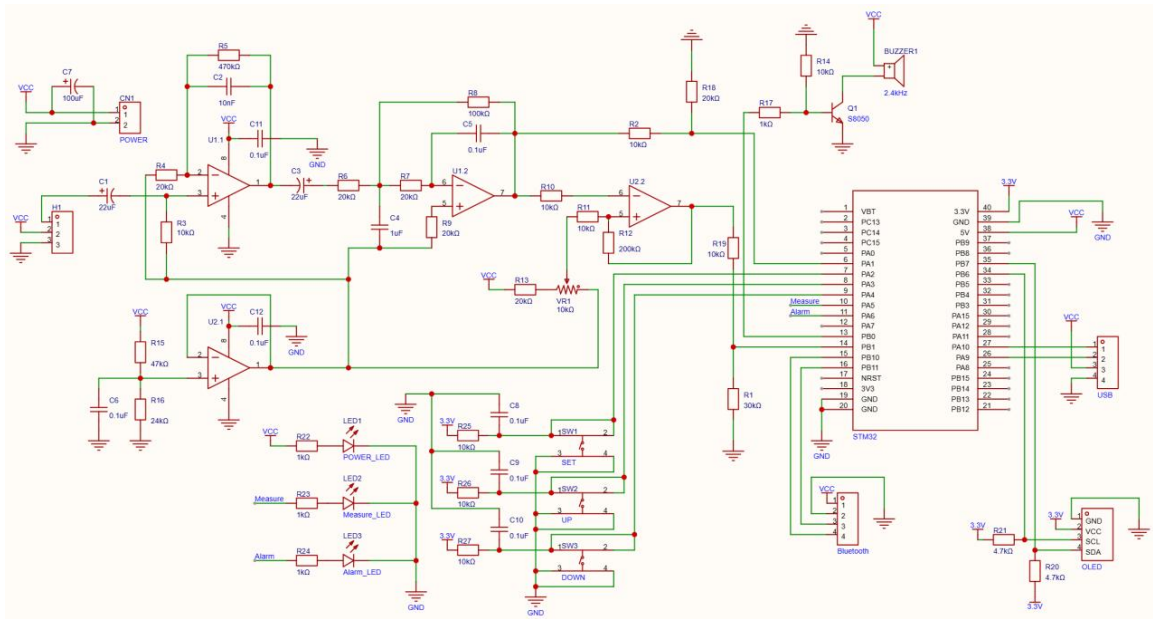


图 2 实验电路图

2、PCB 设计

设计 PCB 时，小组将器件按功能模块布局，STM32 数字电路和模拟部分电路分开，电路板整体均匀布局，最终的 PCB 如图 3 所示。

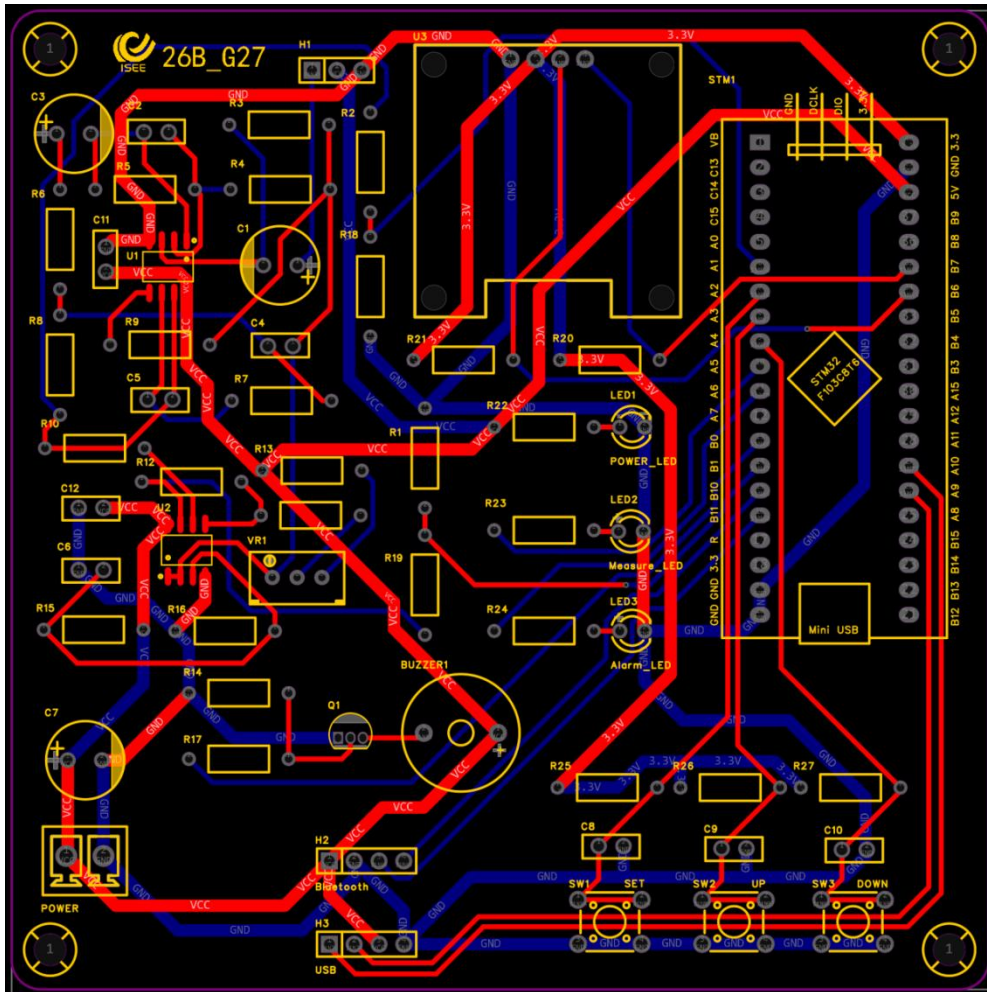


图 3 实验 PCB

3、电路焊接与搭建

将所需器件焊接到加工出的电路板上，并完成各模块的组装，便完成了完整的硬件电路的搭建，最终成果如图 4 所示。

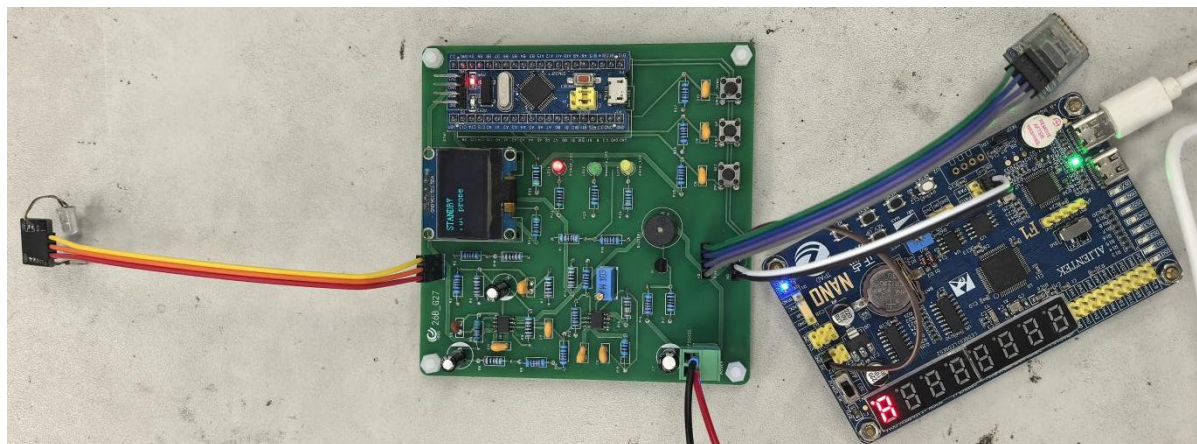


图 4 硬件电路

（二）软件设计

1、软件总体结构

软件采用“定时中断完成实时任务、主循环完成交互任务”的分层结构。TIM2 以 20 ms 为周期触发采样，在中断中完成 ADC 读取、双阈值脉搏计数、按键消抖和采样数据入队；主循环负责系统状态转换、OLED 显示、USB 串口与蓝牙发送以及报警处理。采样与显示、通信相互解耦，避免 OLED 刷新或串口发送阻塞采样过程，从而保证 50 Hz 采样率和 60 s 测量时间的准确性。

便携式脉搏测试仪软件总体流程

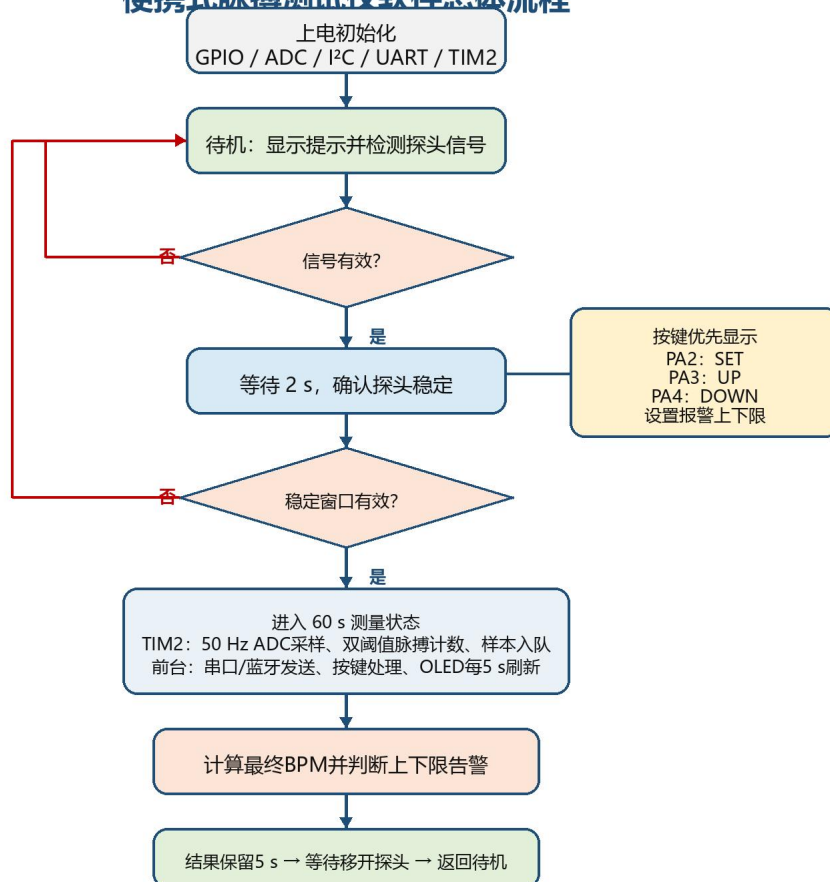


图 5 软件总体流程图

2、功能模块划分

(1) **系统初始化模块**：初始化 GPIO、ADC1、I²C1、USART1、USART3 和 TIM2，并完成 OLED 自检及 I²C 地址扫描。

(2) **信号检测与状态机模块**：在待机状态检测探头信号；信号有效后先稳定 2 s，再进入 60 s 测量；结果保留 5 s 后等待移开探头，防止同一次放置被重复触发。

(3) **采样与脉搏计数模块**：PA1 接收调理后的模拟脉搏波。TIM2 以 50 Hz 触发 ADC 采样，采用高、低双阈值形成迟滞，波形上穿高阈值计一次，回落至低阈值后才允许下一次计数。

(4) **显示与按键模块**：OLED 通过 PB6/PB7 的 I²C1 接口显示状态、实时 BPM、测量时间和报警上下限；PA2、PA3、PA4 分别为 SET、UP、DOWN，按键采用低电平有效并进行 40 ms 消抖。

(5) **通信与数据记录模块**：USART1 以 115200 bit/s 向 PC 连续发送“序号、ADC 值、电压”CSV 数据，便于 MATLAB 实时绘图；USART3 以 9600 bit/s 连接 HC-06，向手机发送精简波形与测量结果。

(6) **报警输出模块**：测量结束后比较最终 BPM 与用户设定上下限，越限时 PB0 驱动 S8050 和有源蜂鸣器，同时 PA6 点亮报警指示灯。PA5 用于测量状态指示，电源指示灯由 VCC 直接供电。

3、脉搏识别与计时方法

为抑制阈值附近噪声造成的重复计数，程序设置高阈值 THRESHOLD_ADC_HIGH=1400 和低阈值 THRESHOLD_ADC_LOW=1200。仅当采样值由低状态上穿高阈值时脉搏计数加 1；随后必须下降到低阈值以下，状态才重新置为“可触发”。该迟滞方法比单阈值判断更稳定。60 s 测量结束时，脉搏总数在数值上即为每分钟脉搏次数；测量过程中则按“已计脉搏数×60000/已测毫秒数”计算实时 BPM。

所有时间判断均基于 HAL_GetTick() 的绝对时间差，而不是累加 HAL_Delay() 次数。测量线程即使执行 OLED 更新或串口发送，也不会改变 60 s 截止时刻；ADC 数据由 128 点环形队列从中断传递到主循环，解决了早期调试中“刷新显示时漏掉一个脉搏”和计时从 25 s 跳到 45 s、70 s 的问题。

4、主要接口与参数

项目	配置	用途
ADC 采样	PA1 / ADC1_IN1, 50 Hz	采集脉搏模拟波形
按键	PA2/PA3/PA4, 低电平有效	SET / UP / DOWN 设置报警限
OLED	PB6-SCL、PB7-SDA, I ² C1	显示状态、BPM 和上下限
PC 串口	PA9-TX、PA10-RX, 115200, 8N1	实时波形和最终结果
蓝牙	PB10-TX、PB11-RX, 9600, 8N1	HC-06 手机无线透传
输出	PA5 测量灯、PA6 报警灯、PB0 蜂鸣器	状态指示与越限报警
时间参数	稳定 2 s；测量 60 s；显示刷新 5 s；结果保留 5 s	自动测量状态机

表 2 主要接口与参数

核心程序由 CubeMX 生成的 HAL 工程扩展完成，完整 main.c 代码见附录；OLED 字库与驱动、CubeMX 工程文件以及 MATLAB 实时显示程序随软件工程包一并提交。

八、实验测试

(一) 硬件电路测试

1、运放相关电路测试

小组首先测试了硬件电路中的运放相关电路，包括两级放大滤波电路（U1.1 和 U1.2）、偏置电路（U2.1）、比较整形电路三部分。

该部分测试输入为信号源产生的频率 2Hz，幅度 10mVpp 的正弦波，用于模拟光电传感器的输出。通过调节电位器 VR_1 ，对比较阈值进行调整，使阈值电压设定在输入信号的幅值范围内，将模拟波形转换为数字脉冲信号。用示波器检测分压后的 A 端口和 B 端口输出，结果如图 6 所示，其中 A 端口输出的模拟信号为黄色信号，B 端口输出的数字信号为蓝色信号。可以发现，A 端口输出了放大后的输入正弦波，B 端口输出了正确的数字脉冲信号，符合设计要求。



图 6 运放相关电路测试结果

2、光电传感器测试

小组接着把光电传感器接入了电路，代替信号源作为输入信号。接通电源，用示波器检测分压后的 A 端口和 B 端口输出，结果如图 7 所示。可以发现，A 端口输出了放大后的输入信号，即光电传感器输出信号，该信号正确反映了人体的脉搏，且无明显的削顶现象，B 端口也输出了有效的数字脉冲信号，硬件电路功能可以正常实现。



图 7 光电传感器测试结果

（二）软件测试

1、测试

程序首先在 Keil MDK 中进行全量编译，编译结果为 0 Error、0 Warning；随后通过 CMSIS-DAP 下载到 STM32F103 核心板。调试时先用串口助手验证 USART1 输出，再依次验证 ADC 采样、OLED 显示、按键设置、蜂鸣报警、HC-06 透传和自动测量状态机。PC 串口采用 115200 bit/s、8 位数据位、无校验、1 位停止位；蓝牙串口采用 9600 bit/s、8N1。

2、采样与计数测试

使用信号源向 PA1 输入带直流偏置的 1.5 Hz 和 2 Hz 周期信号，串口每 20 ms 输出一组 ADC 数据。连续 2 s 可获得 100 个采样点，导入 Excel 及 MATLAB 后能够恢复稳定周期波形。对 1.5 Hz 输入，理论值为 90 BPM；对 2 Hz 输入，理论值为 120 BPM。通过调整高、低阈值并采用中断计数后，计数不再受 OLED 刷新和串口传输影响，测量结果与理论频率基本一致。

3、自动测量与人机交互测试

探头未放置时 OLED 显示 STANDBY；检测到有效信号后显示 Wait 2s，稳定 2s 后自动开始 60 s 测量。测量期间 OLED 每 5 s 刷新一次实时 BPM 和计时，PC 及手机持续接收波形数据。60 s 到达后停止计数并输出最终 BPM，结果在 OLED 上保留 5 s；随后进入“Remove probe”状态，只有撤离并重新放置探头才启动下一次测量。按下 SET 可在低限、高限和退出三种状态间切换，UP、DOWN 改变当前门限；设置界面优先显示，退出后恢复测量界面。

4、软件调试问题与处理

调试现象	原因分析	处理方法及结果
BPM 偏低、刷新时漏计	采样、串口发送、OLED 刷新串行执行，阻塞期间未检测波形	将 50 Hz ADC 采样和计数移入 TIM2 中断，主循环只消费队列和刷新显示，漏计问题消除。
单阈值计数接近 0 或重复触发	阈值与波形幅度不匹配，且阈值附近噪声反复翻转	根据实测波形设置双阈值迟滞，确保每周期只计一次。
显示时间跳变、60 s 不准确	用循环次数或阻塞延时累计时间，通信时间被重复计入	统一使用绝对时间差，测量在真实 60 s 到达时结束。
按键无响应	按键实际接 PA2~PA4 且为外部上拉、低电平有效	按实际原理图重设 GPIO，在 50 Hz 中断中消抖并缓存按键事件，设置操作稳定。

表 3 软件调试问题与处理

综合测试表明，软件能够完成自动启停、60 s 脉搏测量、上下限设置与告警、50 Hz 波形记录、USB 串口传输、MATLAB 实时绘图以及 HC-06 无线透传。最终测试数据及系统运行照片见后续“实验结果”章节。

九、实验结果

1、待机状态

心率计在没有信号输入时会进入待机状态，OLED 显示如图 8 所示。

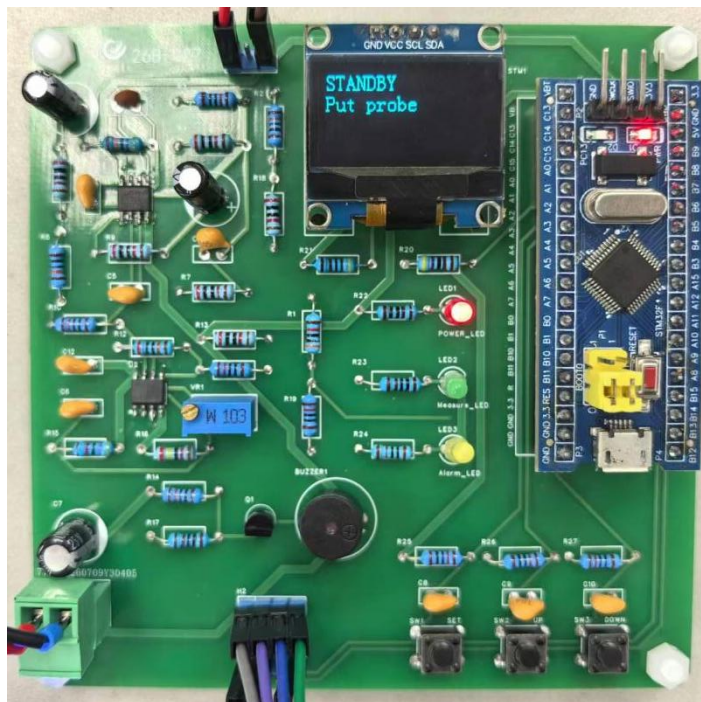


图 8 待机状态

2、准备测量状态

由于探头刚接触测量部位时输入信号可能存在较大的波动，因此脉搏测试仪在发现信号输入时，会先进入 2s 的准备测量阶段，用于等待输入信号稳定，之后再开始测量，此时绿色的测量指示灯亮起，OLED 显示如图 9 所示。

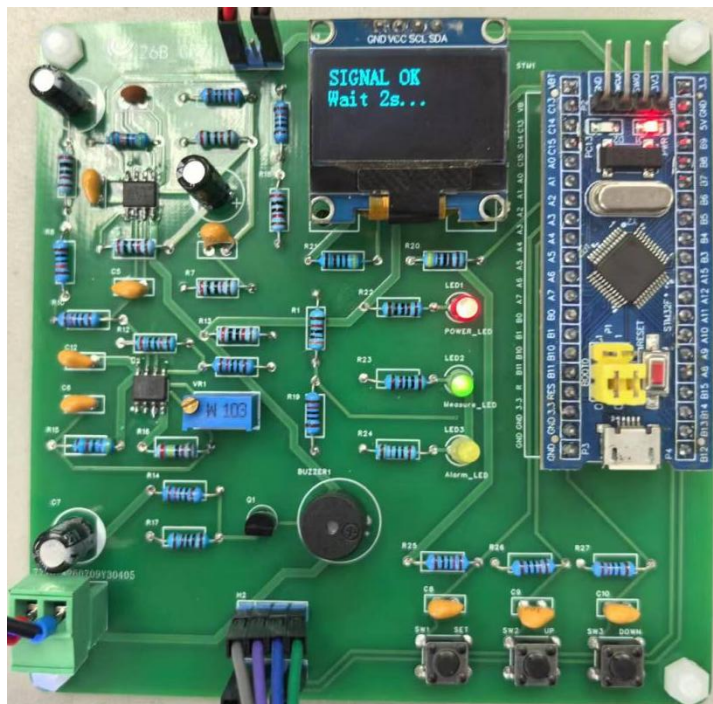


图 9 准备测量状态

3、开始测量状态

准备测量阶段结束后，脉搏测试仪便会开始进行 60s 的脉搏测量，对信号进行计数，OLED 显示如图 10 所示。

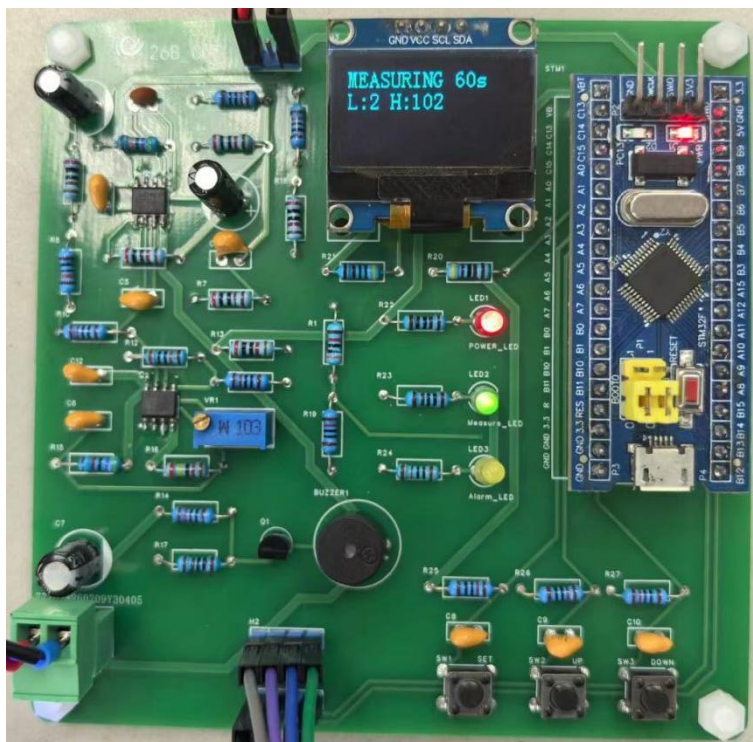


图 10 开始测量状态

4、测量状态

在测量过程中，脉搏测试仪每隔 5s 会输出当前的脉搏值，实现脉搏的快速检测，OLED 显示如图 11 所示。

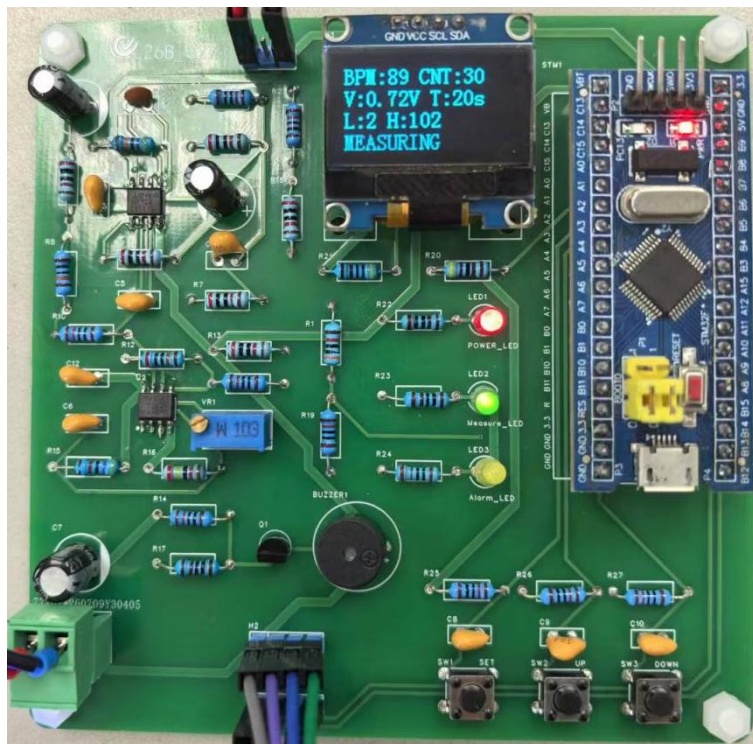


图 11 测量状态

5、结束测量状态

60s 测量结束后，脉搏测试仪会输出最终的脉搏值并保留 5 秒，此时绿色的测量指示灯熄灭，OLED 显示如图 12 所示。

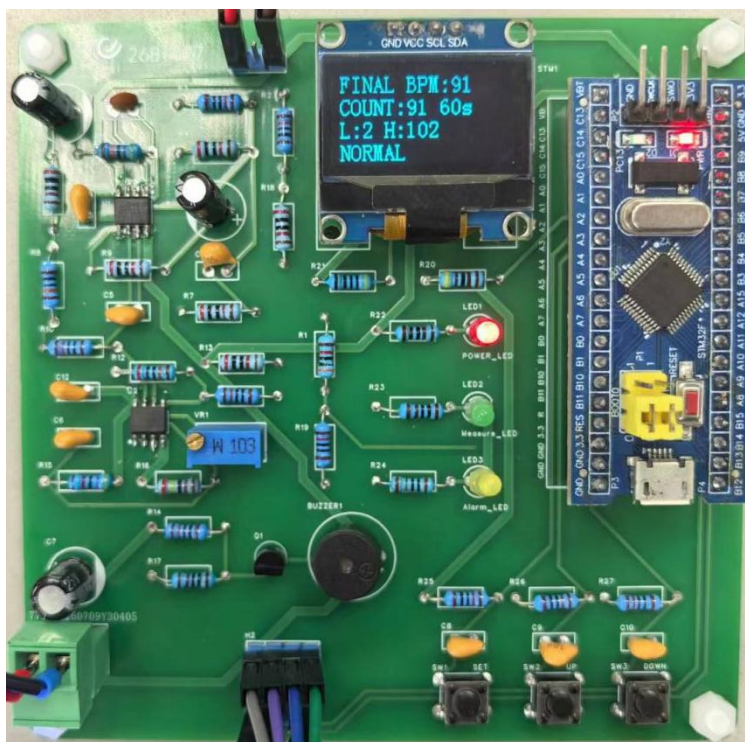


图 12 结束测量状态

6、等待探头移除状态

最终的脉搏值显示结束后，脉搏测试仪会自动进入等待探头移除状态，此时需要撤离探头使其进入待机状态，等待下一次测试，OLED 显示如图 13 所示。

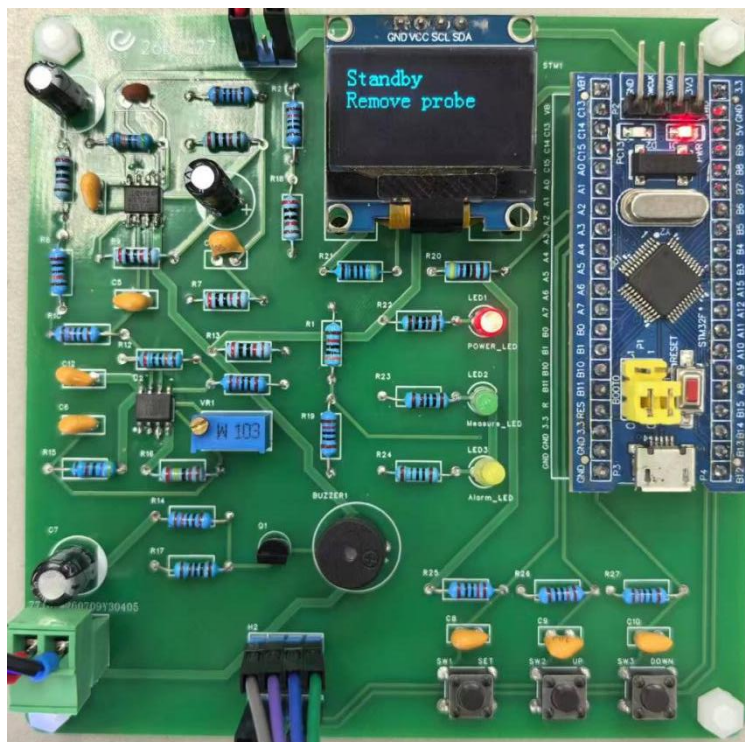


图 13 等待探头移除状态

7、告警状态

60s 测量结束后，若测得的脉搏值不在设定的告警阈值区间内，脉搏测试仪便会进行告警，此时蜂鸣器发出警报，黄色的告警指示灯亮起，OLED 显示如图 14 所示。

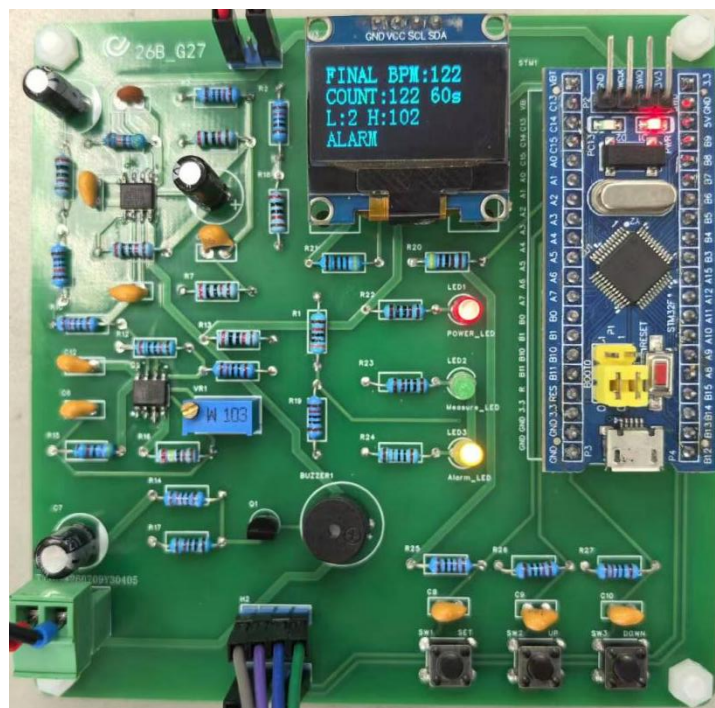


图 14 告警状态

8、告警阈值设置

在测量过程中，脉搏测试仪的三个按键可以设置告警阈值，其中 SET 按键用于进入设置状态和选择调节上限或下限，UP 按键用于增大数值，DOWN 按键用于减小数值，告警阈值设置时 OLED 显示如图 15 所示。

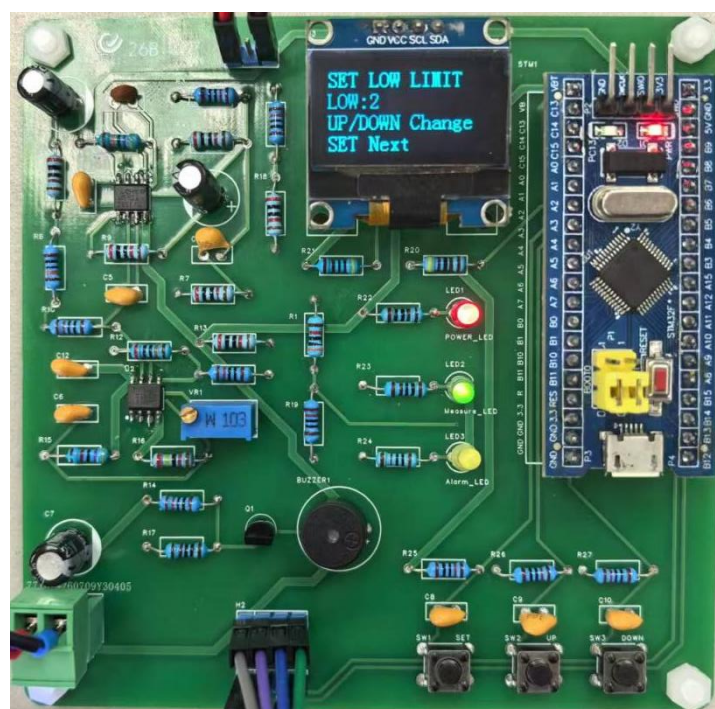


图 15 告警阈值设置

9、蓝牙串口

依靠蓝牙透传模块的无线传输，脉搏测试仪可以与手机连接，通过手机串口助手实时接收脉搏数据，并在测量结束后接收测量结果，手机界面显示如图 16 所示。

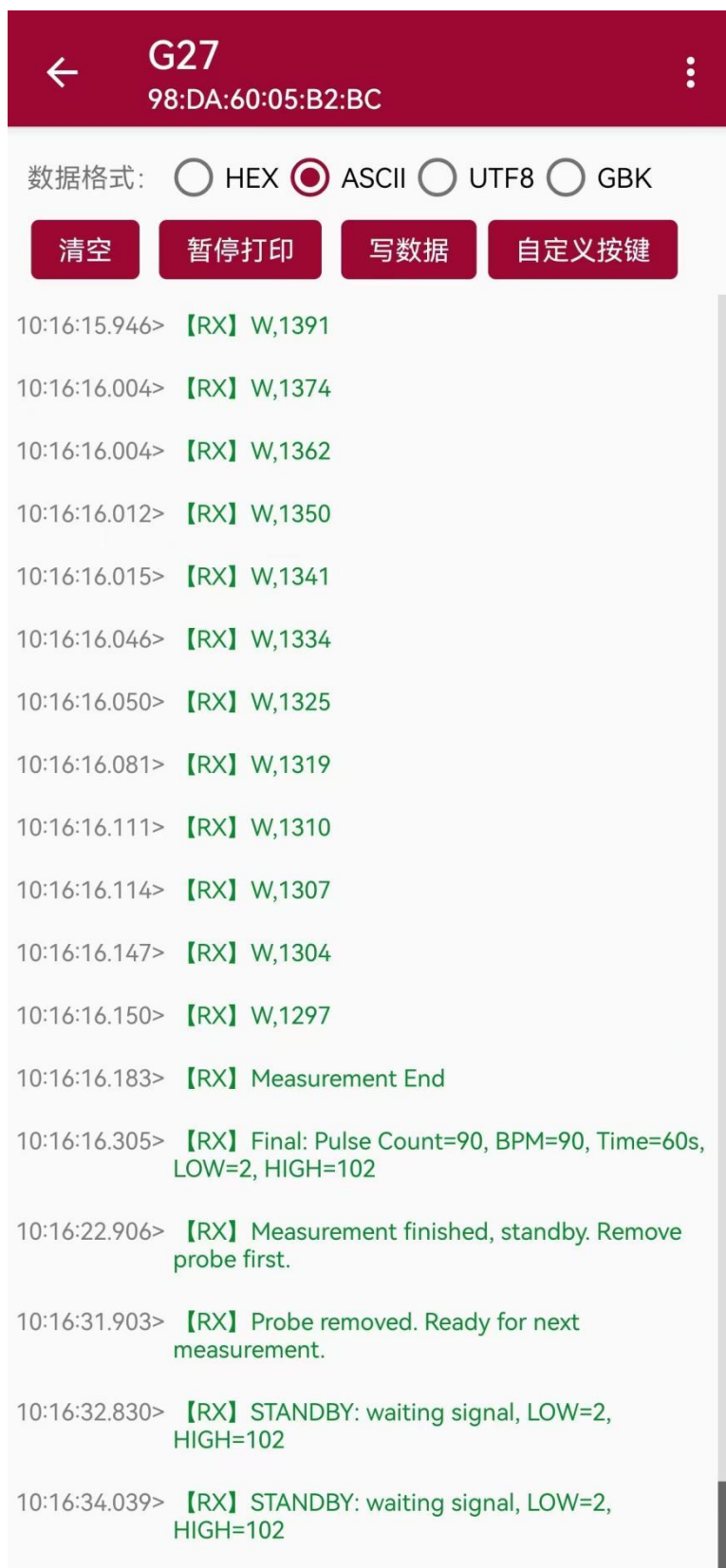


图 16 蓝牙串口

10、电脑波形实时绘制

依靠 STM32 开发板上的 USB 转串口模块，脉搏测试仪可以与电脑连接，传输波形信号数据。利用小组编写的 MATLAB 程序，可以利用串口传输的数据进行实时脉搏波形绘制，波形绘制界面如图 17 所示。

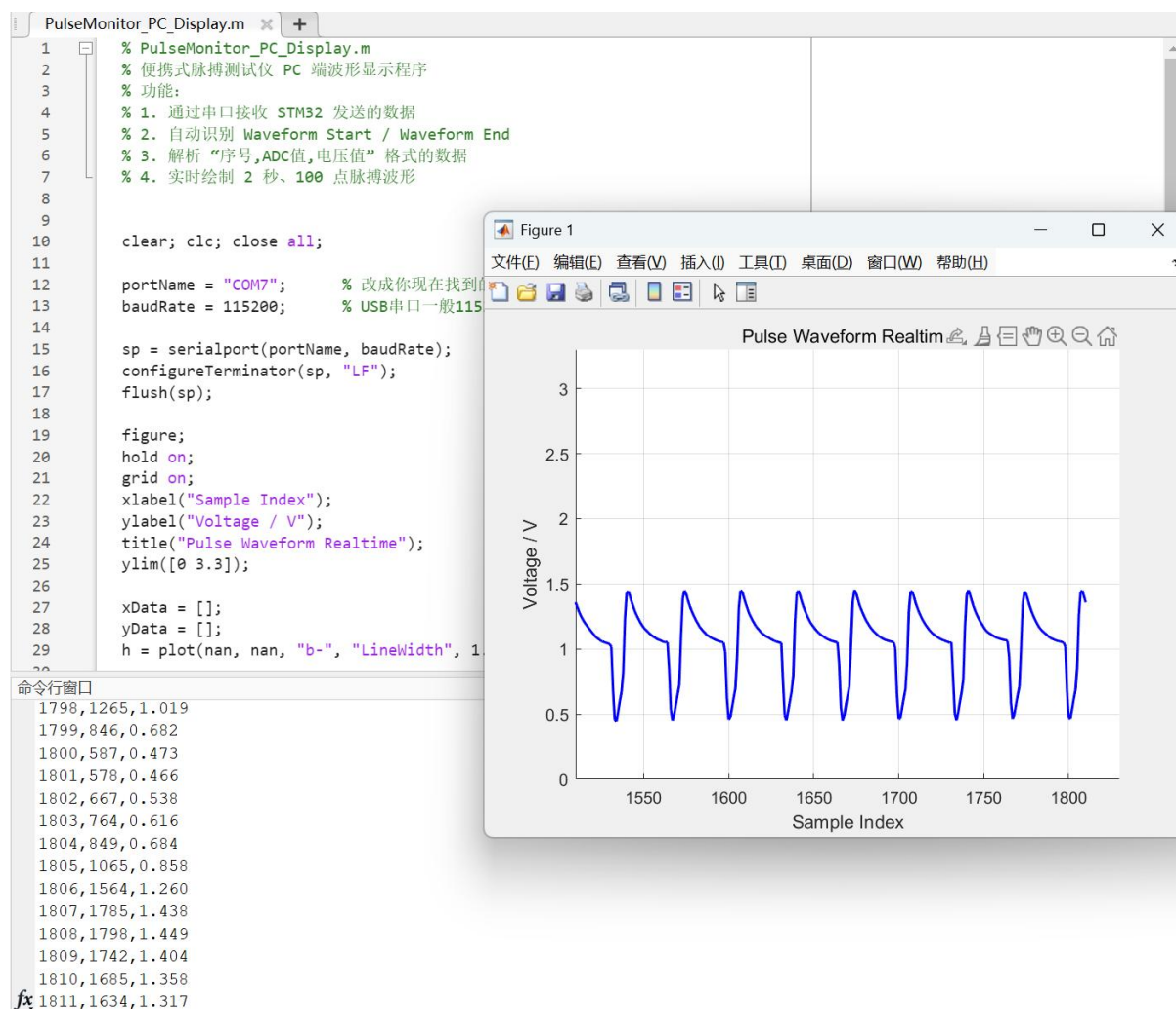


图 17 波形绘制

十、小组分工与工作量

小组分工与工作量如表 4 所示。

姓名	分工	贡献比
	电路设计、开题 PPT、开题答辩、电路图绘制、PCB 绘制、电路搭建、软硬件调试、总结报告编写	50%
	电路设计、开题 PPT、电路图绘制、PCB 绘制、编程作业、代码设计、软硬件调试、总结报告编写	50%

表 4 小组分工与工作量

十一、实验小结与心得体会

1、个人总结

在本次短学期课程中，我主要负责电路设计、开题 PPT 制作、开题答辩、电路图绘制、PCB 版图绘制、硬件电路搭建与焊接，以及后续的软硬件联合调试和总结报告编写。项目启动的第一天，

小组经过研究比较，确定了“便携式脉搏测试仪”这一课题，目标是设计一个能够从指尖采集脉搏信号、实时显示心率、并具备自动测量和无线传输功能的完整系统。作为硬件部分的主要负责人，我需要将抽象的生理信号检测原理转化为可靠的实际电路，并确保其与 STM32 控制器的接口完美匹配。

在方案设计阶段，我仔细研读了光电脉搏检测的基本原理——利用血液灌注变化引起组织半透明度改变，从而调制光强信号。但光电接收管输出的信号仅为毫伏级，且叠加有环境光干扰、手指抖动噪声以及高频电路噪声，因此信号调理电路是成败的关键。我决定采用两级放大滤波结构，并在反馈回路中加入了电容，成功限制了带宽并滤除了工频干扰和高频噪声。偏置电路的设计尤为关键，因为 LM358 运放采用 5V 单电源供电，无法处理负电压信号，我通过电阻分压将偏置电压设定为 1.69V，使信号在放大后能完整地落在运放的线性区内，避免双向削顶。比较器部分则采用迟滞比较器，通过电位器调节阈值电压，将模拟脉搏波整形为干净的数字脉冲，同时加入分压电阻保护单片机 I/O 口。这些参数并非一次成功，最初计算时忽略了运放输出摆幅限制，导致波形顶部被削平，后来我重新调整了放大倍数和偏置电压，并在示波器上逐级验证，最终得到了理想的波形。

电路图绘制和 PCB 设计是我投入精力最多的环节。考虑到模拟信号对噪声极为敏感，我严格遵循了模拟地与数字地单点连接的原则，将传感器接口、放大滤波电路和比较器布置在 PCB 的一侧，而 STM32、蓝牙模块和 OLED 等数字电路布置在另一侧，电源线和信号线的宽度、间距也经过反复考量。在绘制过程中，我曾因布局不合理导致部分走线冲突，不得不重新调整原理图并再次布局走线，但这反而让我对嘉立创 EDA 的操作更加熟练，也学会了如何从整体功能分区出发优化布局。最终，当 PCB 板加工回来后，我仔细焊接了所有贴片元件和直插件，包括 LM358、电位器、排针等，焊接完成后用万用表检查了电源短路和断路情况，确保每一路供电正常。

硬件调试是最考验耐心的阶段。上电后，我先用信号源产生 2Hz、10mVpp 的正弦波模拟传感器输出，逐级测量 A 端口和 B 端口的波形。过程中，我反复调节电位器，找到了最稳定的阈值点。此外，我还配合组员进行了软件调试，对程序存在的问题进行排查。在发挥部分的实现中，我协助完成了蓝牙模块和 USB 转串口模块的硬件连接，确保 UART 电平匹配和通信稳定。我们还利用 STM32 的 ADC 以 50Hz 采样率采集脉搏波形，并通过串口发送到 PC。

回顾整个实验过程，我最大的收获是深刻理解了理论指导实践，实践反哺理论的含义。课堂上学到的运放放大、滤波、比较器等知识，在真实电路中会因为器件非理想性、电源噪声、布局寄生效应等因素而产生偏差，必须通过示波器观测和参数微调才能达到最佳性能。同时，我也认识到硬件设计不能孤立进行，必须与软件工程师紧密沟通，比如比较器输出的脉冲幅度、占空比、上升沿时间等参数直接影响单片机的捕获精度。这次项目锻炼了我解决工程问题的能力，从计算选型、绘制图纸、焊接调试到系统联调，每一步都充满了挑战，但每一步的突破都带来巨大的成就感。此外，团队协作让我学会了如何高效分工、如何记录实验日志以便追溯问题，这些习惯将对我未来的科研和工程实践大有帮助。虽然我们在调试过程中经历过多次返工，但看到最终成品能够稳定工作、波形清晰地显示在电脑屏幕上时，所有的辛苦都化为喜悦。我相信，这段宝贵的经历将成为我继续深入学习电子系统设计的坚实基础。

2、个人总结

在本次“便携式脉搏测试仪”课程设计中，我主要负责 STM32 软件方案设计、CubeMX 外设配置、Keil 程序编写与下载、OLED 和按键人机界面、USB 串口及 HC-06 蓝牙通信、MATLAB 实时波形显示，并配合完成整机软硬件联调和测试数据整理。从最初把各功能分别点亮，到最后把探头检测、2 s 稳定、60 s 测量、5 s 结果保持和等待移开探头整合成完整状态机，我对嵌入式系统有了更加具体的认识。

软件调试中最突出的问题是实时性。初版程序在主循环里依次进行 ADC 采样、串口发送、OLED 刷新和延时，单独测试时均能工作，但合并后出现刷新期间漏掉脉搏、BPM 偏低以及计时从 25 s 跳到 45 s、70 s 等现象。通过示波器、信号源和串口日志逐项排查，我把 50 Hz 采样、双阈值计数

和按键扫描放入 TIM2 中断，将采样值写入环形队列；主循环只负责通信、显示和状态转换，并用绝对时间差判断 5 s 刷新和 60 s 结束。修改后，显示与通信不再顶掉采样任务，系统计时和脉搏计数明显稳定。这一过程使我真正理解了中断实时任务与前台非实时任务分离的重要性。

在脉搏算法方面，我先采用单阈值上升沿计数，但实际波形在阈值附近存在噪声，容易出现重复计数或完全不触发。结合 MATLAB 实时曲线和 ADC 原始数据，我改为高、低双阈值迟滞判定：上穿高阈值计数，回落到低阈值后才允许下一次触发，并根据实测幅度调整阈值。与此同时，我完成了 50 Hz、2 s 共 100 点的波形记录和 CSV 输出，使 PC 端能够直接绘制波形，也方便对比输入频率、BPM 结果和误差来源。这个过程让我体会到算法参数不能只靠理论给定，必须通过真实数据进行验证。

人机交互和接口联调同样锻炼了我的细致程度。OLED 不亮时，我从供电、PB6/PB7 接线、4.7 k Ω 上拉、I²C 地址和初始化顺序逐项检查；按键无响应时，重新核对原理图，确认 PA2、PA3、PA4 分别对应 SET、UP、DOWN 且为低电平有效；蓝牙通信则同时核对 PB10/PB11 收发交叉、9600 bit/s 波特率和共地。通过这些问题，我形成了“先确认电源与接口，再验证单模块，最后联合运行”的调试习惯，也认识到软件必须严格以实际原理图和器件特性为依据。

本次项目让我把模拟电路、单片机外设、实时程序、串口通信和上位机数据处理串联成了一个完整系统。与组员协作时，我们及时共享引脚表、波形截图和调试记录，使硬件参数修改与软件阈值调整能够相互配合。后续若继续完善，我会采用定时器触发 ADC 并结合 DMA 进一步降低中断负担，加入自适应基线和动态阈值算法，同时增加多组受试者标定和功耗测试。虽然调试过程经历了多次返工，但最终看到 OLED、串口、蓝牙和报警功能按照状态机稳定运行，我对电子系统设计的整体性、规范性和工程验证方法有了更深理解。

十二、附录：软件代码（main.c）

说明：以下为本系统 STM32 端完整 main.c 源文件。OLED 驱动、字库、CubeMX 工程和 MATLAB 实时显示程序随软件工程包另行提交。

```
0001  /* USER CODE BEGIN Header */
0002  /**
0003
0004  ****
0005  * @file          : main.c
0006  * @brief         : 便携式脉搏测试仪主程序
0007  ****
0008  * @attention 注意
0009  *
0010  * Copyright (c) 2026 STMicroelectronics.
0011  * All rights reserved.
0012  * This software is licensed under terms that can be found in the LICENSE file
0013  * in the root directory of this software component.
0014  * If no LICENSE file comes with this software, it is provided AS-IS.
0015  *
0016  ****
0017  */
```

```

0018  /* USER CODE END Header */
0019  /* 头文件 -----*/
0020  #include "main.h"
0021  #include "adc.h"
0022  #include "i2c.h"
0023  #include "usart.h"
0024  #include "gpio.h"
0025
0026  /* 私有头文件 -----*/
0027  /* USER CODE BEGIN Includes */
0028  #include <stdio.h>
0029  #include <string.h>
0030  /* USER CODE END Includes */
0031
0032  /* 私有类型定义 -----*/
0033  /* USER CODE BEGIN PTD */
0034
0035  /* USER CODE END PTD */
0036
0037  /* 私有宏定义 -----*/
0038  /* USER CODE BEGIN PD */
0039  #define THRESHOLD_ADC_HIGH 1400
0040  #define THRESHOLD_ADC_LOW 1200
0041  #define THRESHOLD_ADC THRESHOLD_ADC_HIGH
0042  #define BPM_LOW_LIMIT_DEFAULT 0
0043  #define BPM_HIGH_LIMIT_DEFAULT 100
0044  #define THRESHOLD_VALID_DIFF 100
0045  #define ADC_QUEUE_SIZE 128
0046  #define KEY_EVENT_SET 0x01U
0047  #define KEY_EVENT_UP 0x02U
0048  #define KEY_EVENT_DOWN 0x04U
0049  /* USER CODE END PD */
0050
0051  /* 私有宏 -----*/
0052  /* USER CODE BEGIN PM */
0053
0054  /* USER CODE END PM */
0055
0056  /* 私有变量 -----*/
0057
0058  /* USER CODE BEGIN PV */
0059  int bpm_low_limit = BPM_LOW_LIMIT_DEFAULT;
0060  int bpm_high_limit = BPM_HIGH_LIMIT_DEFAULT;
0061  int key_mode = 0; /* 0: 正常显示, 1: 设置下限, 2: 设置上限 */

```

```

0062
0063 // ===== 与 TIM2 采样中断共享的运行数据 =====
0064 volatile uint16_t latest_adc = 0;    // 最新 ADC 采样值
0065 volatile uint8_t  adc_ready = 0;     // 新的 ADC 采样值准备完成标志
0066 volatile uint8_t measurement_active = 0;
0067 volatile uint16_t isr_pulse_count = 0;
0068 volatile uint8_t isr_pulse_high = 0;
0069 volatile uint16_t adc_queue[ADC_QUEUE_SIZE];
0070 volatile uint16_t adc_queue_head = 0;
0071 volatile uint16_t adc_queue_tail = 0;
0072 volatile uint16_t adc_queue_overflow = 0;
0073 volatile uint8_t key_event_flags = 0;
0074 /* USER CODE END PV */
0075
0076 /* 私有函数声明 -----*/
0077 void SystemClock_Config(void);
0078 /* USER CODE BEGIN PFP */
0079 void OLED_Init(void);
0080 void OLED_Clear(void);
0081 void OLED_P8x16Str(unsigned char x, unsigned char y, unsigned char ch[]);
0082 void Send_To_All(char *msg);
0083 void Send_Wave_Sample(uint32_t index, uint16_t adc_value);
0084 uint16_t Read_ADC1(void);
0085 uint8_t Signal_Is_Valid(void);
0086 void Wait_Signal_Remove(void);
0087 void Key_Process(void);
0088 void I2C_Scan(void);
0089 void OLED_Show_Key_Setting(void);
0090 void Key_Scan_20ms_ISR(void);
0091 /* USER CODE END PFP */
0092
0093 /* 用户代码 -----*/
0094 /* USER CODE BEGIN 0 */
0095 void Send_To_All(char *msg)
0096 {
0097     HAL_UART_Transmit(&huart1, (uint8_t *)msg, strlen(msg), 100);
0098     HAL_UART_Transmit(&huart3, (uint8_t *)msg, strlen(msg), 100);
0099 }
0100
0101 /*
0102  * USART1 以 115200 bit/s 向 PC 发送完整 CSV 记录。
0103  * HC-06 工作在 9600 bit/s，因此只接收精简后的波形帧。
0104  * 这样可以确保前台发送时间短于 20 ms 采样周期，
0105  * 避免串口数据堆积影响 60 秒测量截止时间。

```

```

0106  */
0107 void Send_Wave_Sample(uint32_t index, uint16_t adc_value)
0108 {
0109     char pc_msg[48];
0110     char bt_msg[20];
0111     uint32_t millivolts;
0112
0113     millivolts = ((uint32_t)adc_value * 3300UL + 2047UL) / 4095UL;
0114
0115     sprintf(pc_msg, "%lu,%u,%lu.%03lu\r\n",
0116             (unsigned long)index,
0117             (unsigned int)adc_value,
0118             (unsigned long)(millivolts / 1000UL),
0119             (unsigned long)(millivolts % 1000UL));
0120     HAL_UART_Transmit(&huart1, (uint8_t *)pc_msg, strlen(pc_msg), 100);
0121
0122     sprintf(bt_msg, "W,%u\r\n", (unsigned int)adc_value);
0123     HAL_UART_Transmit(&huart3, (uint8_t *)bt_msg, strlen(bt_msg), 100);
0124 }
0125
0126 uint16_t Read_ADC1(void)
0127 {
0128     uint16_t value;
0129     HAL_ADC_Start(&hadc1);
0130     HAL_ADC_PollForConversion(&hadc1, 10);
0131     value = HAL_ADC_GetValue(&hadc1);
0132     HAL_ADC_Stop(&hadc1);
0133     return value;
0134 }
0135
0136 uint8_t Signal_Is_Valid(void)
0137 {
0138     uint16_t adc_value;
0139     uint16_t min_adc = 4095;
0140     uint16_t max_adc = 0;
0141     int i;
0142
0143     for (i = 0; i < 20; i++)
0144     {
0145         adc_value = Read_ADC1();
0146         if (adc_value < min_adc) min_adc = adc_value;
0147         if (adc_value > max_adc) max_adc = adc_value;
0148         HAL_Delay(20);
0149     }

```

```

0150
0151     if ((max_adc - min_adc) >= THRESHOLD_VALID_DIFF)
0152     {
0153         return 1;
0154     }
0155     return 0;
0156 }
0157
0158 void Wait_Signal_Remove(void)
0159 {
0160     char msg[80];
0161
0162     OLED_Clear();
0163     OLED_P8x16Str(0, 0, (unsigned char *)"Standby");
0164     OLED_P8x16Str(0, 2, (unsigned char *)"Remove probe");
0165     sprintf(msg, "Measurement finished, standby. Remove probe first.\r\n");
0166     Send_To_All(msg);
0167
0168     while (Signal_Is_Valid())
0169     {
0170         Key_Process();
0171         HAL_Delay(200);
0172     }
0173
0174     OLED_Clear();
0175     OLED_P8x16Str(0, 0, (unsigned char *)"Ready");
0176     OLED_P8x16Str(0, 2, (unsigned char *)"Put finger");
0177     sprintf(msg, "Probe removed. Ready for next measurement.\r\n");
0178     Send_To_All(msg);
0179 }
0180
0181 void OLED_Show_Key_Setting(void)
0182 {
0183     char msg[32];
0184
0185     OLED_Clear();
0186     if (key_mode == 1)
0187     {
0188         OLED_P8x16Str(0, 0, (unsigned char *)"SET LOW LIMIT");
0189         sprintf(msg, "LOW:%d", bpm_low_limit);
0190         OLED_P8x16Str(0, 2, (unsigned char *)msg);
0191         OLED_P8x16Str(0, 4, (unsigned char *)"UP/DOWN Change");
0192         OLED_P8x16Str(0, 6, (unsigned char *)"SET Next");
0193     }

```

```

0194     else if (key_mode == 2)
0195     {
0196         OLED_P8x16Str(0, 0, (unsigned char *)"SET HIGH LIMIT");
0197         sprintf(msg, "HIGH:%d", bpm_high_limit);
0198         OLED_P8x16Str(0, 2, (unsigned char *)msg);
0199         OLED_P8x16Str(0, 4, (unsigned char *)"UP/DOWN Change");
0200         OLED_P8x16Str(0, 6, (unsigned char *)"SET Exit");
0201     }
0202     else
0203     {
0204         OLED_P8x16Str(0, 0, (unsigned char *)"LIMIT SET DONE");
0205         sprintf(msg, "L:%d H:%d", bpm_low_limit, bpm_high_limit);
0206         OLED_P8x16Str(0, 2, (unsigned char *)msg);
0207         OLED_P8x16Str(0, 4, (unsigned char *)"Measuring...");
0208     }
0209 }
0210
0211 void Key_Process(void)
0212 {
0213     uint8_t events;
0214
0215     /* 50 Hz 定时中断负责捕获短按键事件，此处读取并处理按键事件。 */
0216     __disable_irq();
0217     events = key_event_flags;
0218     key_event_flags = 0U;
0219     __enable_irq();
0220
0221     if ((events & KEY_EVENT_SET) != 0U)
0222     {
0223         key_mode++;
0224         if (key_mode > 2) key_mode = 0;
0225         OLED_Show_Key_Setting();
0226     }
0227
0228     if ((events & KEY_EVENT_UP) != 0U)
0229     {
0230         if (key_mode == 1 && bpm_low_limit < bpm_high_limit - 1) bpm_low_limit++;
0231         if (key_mode == 2 && bpm_high_limit < 250) bpm_high_limit++;
0232         if (key_mode != 0) OLED_Show_Key_Setting();
0233     }
0234
0235     if ((events & KEY_EVENT_DOWN) != 0U)
0236     {
0237         if (key_mode == 1 && bpm_low_limit > 0) bpm_low_limit--;

```

```

0238         if (key_mode == 2 && bpm_high_limit > bpm_low_limit + 1) bpm_high_limit--;
0239         if (key_mode != 0) OLED_Show_Key_Setting();
0240     }
0241 }
0242
0243 /* 每 20 ms 对 PA2、PA3、PA4 按键消抖，按键按下时为低电平。 */
0244 void Key_Scan_20ms_ISR(void)
0245 {
0246     static uint8_t low_count[3] = {0U, 0U, 0U};
0247     static uint8_t high_count[3] = {0U, 0U, 0U};
0248     static uint8_t pressed_state = 0U;
0249     static const uint16_t pins[3] = {GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_4};
0250     static const uint8_t masks[3] = {KEY_EVENT_SET, KEY_EVENT_UP,
KEY_EVENT_DOWN};
0251     uint8_t k;
0252
0253     for (k = 0U; k < 3U; k++)
0254     {
0255         if ((GPIOA->IDR & pins[k]) == 0U)
0256         {
0257             high_count[k] = 0U;
0258             if (low_count[k] < 2U) low_count[k]++;
0259             if ((low_count[k] >= 2U) && ((pressed_state & masks[k]) == 0U))
0260             {
0261                 pressed_state |= masks[k];
0262                 key_event_flags |= masks[k];
0263             }
0264         }
0265         else
0266         {
0267             low_count[k] = 0U;
0268             if (high_count[k] < 2U) high_count[k]++;
0269             if (high_count[k] >= 2U)
0270             {
0271                 pressed_state &= (uint8_t)(~masks[k]);
0272             }
0273         }
0274     }
0275 }
0276
0277 void I2C_Scan(void)
0278 {
0279     char msg[64];
0280     uint8_t found = 0;

```

```

0281
0282     Send_To_All("I2C Scan Start\r\n");
0283
0284     for (uint8_t addr = 1; addr < 127; addr++)
0285     {
0286         if (HAL_I2C_IsDeviceReady(&hi2c1, addr << 1, 2, 10) == HAL_OK)
0287         {
0288             sprintf(msg, "I2C Found: 0x%02X, WriteAddr=0x%02X\r\n", addr, addr << 1);
0289             Send_To_All(msg);
0290             found = 1;
0291         }
0292     }
0293
0294     if (!found)
0295     {
0296         Send_To_All("I2C No Device Found\r\n");
0297     }
0298 }
0299
0300 /* TIM2 作为独立的 50 Hz 采样时钟。
0301  * 直接使用外设寄存器配置，避免依赖 CubeMX 生成的 TIM HAL 模块。
0302  */
0303 void TIM2_IRQHandler(void)
0304 {
0305     if ((TIM2->SR & TIM_SR_UIF) != 0U)
0306     {
0307         uint16_t next_head;
0308
0309         TIM2->SR &= ~TIM_SR_UIF;
0310
0311         /* 在待机状态及 OLED/I2C 操作期间仍持续扫描按键。 */
0312         Key_Scan_20ms_ISR();
0313
0314         if (!measurement_active)
0315         {
0316             return;
0317         }
0318
0319         HAL_ADC_Start(&hadc1);
0320         HAL_ADC_PollForConversion(&hadc1, 1);
0321         latest_adc = HAL_ADC_GetValue(&hadc1);
0322         HAL_ADC_Stop(&hadc1);
0323
0324         /* 在中断中统计每个脉搏，避免 OLED 或串口处理造成漏计。 */

```

```

0325         if ((!isr_pulse_high) && (latest_adc >= THRESHOLD_ADC_HIGH))
0326         {
0327             isr_pulse_high = 1;
0328             isr_pulse_count++;
0329         }
0330         else if (isr_pulse_high && (latest_adc <= THRESHOLD_ADC_LOW))
0331         {
0332             isr_pulse_high = 0;
0333         }
0334
0335         /* 将全部采样值放入队列，交由主循环发送。 */
0336         next_head = (uint16_t)((adc_queue_head + 1U) % ADC_QUEUE_SIZE);
0337         if (next_head != adc_queue_tail)
0338         {
0339             adc_queue[adc_queue_head] = latest_adc;
0340             adc_queue_head = next_head;
0341         }
0342         else
0343         {
0344             adc_queue_overflow++;
0345         }
0346
0347         adc_ready = 1;
0348     }
0349 }
0350
0351 // ===== TIM2 50 Hz 采样定时器初始化 =====
0352 static void MX_TIM2_Init(void)
0353 {
0354     __HAL_RCC_TIM2_CLK_ENABLE();
0355
0356     TIM2->CR1 = 0U;
0357     TIM2->PSC = 8000U - 1U;      /* 8 MHz 分频到 1 kHz。 */
0358     TIM2->ARR = 20U - 1U;        /* 1 kHz 进一步分频到 50 Hz。 */
0359     TIM2->CNT = 0U;
0360     TIM2->EGR = TIM_EGR_UG;
0361     TIM2->SR = 0U;
0362     TIM2->DIER = TIM_DIER_UIE;
0363
0364     HAL_NVIC_SetPriority(TIM2_IRQn, 0, 0);
0365     HAL_NVIC_EnableIRQ(TIM2_IRQn);
0366
0367     TIM2->CR1 = TIM_CR1_CEN;
0368 }

```

```

0369  /* USER CODE END 0 */
0370
0371  /**
0372   * @brief  程序入口函数
0373   * @retval int 返回值
0374   */
0375  int main(void)
0376  {
0377      /* USER CODE BEGIN 1 */
0378
0379      /* USER CODE END 1 */
0380
0381      /* 单片机配置 -----*/
0382
0383      /* 复位全部外设，并初始化 Flash 接口和系统滴答定时器。 */
0384      HAL_Init();
0385
0386      /* USER CODE BEGIN Init */
0387
0388      /* USER CODE END Init */
0389
0390      /* 配置系统时钟。 */
0391      SystemClock_Config();
0392
0393      /* USER CODE BEGIN SysInit */
0394
0395      /* USER CODE END SysInit */
0396
0397      /* 初始化全部已配置外设。 */
0398      MX_GPIO_Init();
0399      MX_ADC1_Init();
0400      MX_I2C1_Init();
0401      MX_USART1_UART_Init();
0402      MX_USART3_UART_Init();
0403      HAL_Delay(500);
0404
0405      // ===== 启动 50 Hz 定时中断采样 =====
0406      MX_TIM2_Init();
0407
0408      I2C_Scan();
0409      OLED_Init();
0410      HAL_Delay(100);
0411      OLED_Clear();
0412      OLED_P8x16Str(0, 0, (unsigned char *)"Pulse Monitor");

```

```

0413     OLED_P8x16Str(0, 2, (unsigned char *)"OLED OK");
0414     /* PA4 作为减小键使用，不能再将其作为 POWER_LED 输出。 */
0415     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); /* 关闭蜂鸣器 */
0416     /* USER CODE BEGIN 2 */
0417
0418     /* USER CODE END 2 */
0419
0420     /* 主循环。 */
0421     /* USER CODE BEGIN WHILE */
0422     while (1)
0423     {
0424         /* USER CODE BEGIN 3 */
0425         uint16_t adc_value = 0;
0426         float voltage = 0.0f;
0427         char msg[96];
0428         int i = 0;
0429         uint16_t count_snapshot = 0;
0430         uint16_t queued_adc = 0;
0431         int bpm_live = 0;
0432         int bpm_final = 0;
0433         uint32_t start_tick = 0;
0434         uint32_t elapsed_ms = 0;
0435         uint32_t last_oled_tick = 0;
0436         uint8_t is_alarm = 0;
0437         uint8_t stable_ok = 1;
0438         uint16_t stable_adc = 0;
0439         uint16_t stable_min = 4095;
0440         uint16_t stable_max = 0;
0441
0442         Key_Process();
0443
0444         /* 未检测到有效脉搏信号：保持待机并继续等待。 */
0445         if (!Signal_Is_Valid())
0446         {
0447             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); /* 关闭测量指示
灯 */
0448             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET); /* 关闭告警指示
灯 */
0449             HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); /* 关闭蜂鸣器 */
0450             if (key_mode != 0)
0451             {
0452                 OLED_Show_Key_Setting();
0453             }
0454             else

```

```

0455     {
0456         OLED_Clear();
0457         OLED_P8x16Str(0, 0, (unsigned char *)"STANDBY");
0458         OLED_P8x16Str(0, 2, (unsigned char *)"Put probe");
0459     }
0460     sprintf(msg, "STANDBY: waiting signal, LOW=%d, HIGH=%d\r\n", bpm_low_limit,
bpm_high_limit);
0461     Send_To_All(msg);
0462     HAL_Delay(300);
0463     continue;
0464 }
0465
0466 /* 检测到信号后等待 2 秒，使手指和探头接触稳定。 */
0467 OLED_Clear();
0468 OLED_P8x16Str(0, 0, (unsigned char *)"SIGNAL OK");
0469 OLED_P8x16Str(0, 2, (unsigned char *)"Wait 2s...");
0470 Send_To_All("Signal detected, wait 2s before measurement\r\n");
0471 stable_ok = 1;
0472 stable_min = 4095;
0473 stable_max = 0;
0474 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET); /* 点亮测量指示灯
*/
0475
0476 /* 按 20 ms 间隔采集 100 点，构成准确的 2 秒稳定等待时间。 */
0477 for (i = 0; i < 100; i++)
0478 {
0479     stable_adc = Read_ADC1();
0480     if (stable_adc < stable_min) stable_min = stable_adc;
0481     if (stable_adc > stable_max) stable_max = stable_adc;
0482     Key_Process();
0483     HAL_Delay(20);
0484 }
0485
0486 if ((stable_max - stable_min) < THRESHOLD_VALID_DIFF)
0487 {
0488     Send_To_All("Signal lost during 2s wait, back to standby\r\n");
0489     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
0490     OLED_Clear();
0491     OLED_P8x16Str(0, 0, (unsigned char *)"SIGNAL LOST");
0492     OLED_P8x16Str(0, 2, (unsigned char *)"Put probe");
0493     HAL_Delay(500);
0494     stable_ok = 0;
0495 }
0496

```

```

0497     if (!stable_ok)
0498     {
0499         continue;
0500     }
0501
0502     /* ===== 开始 60 秒正式测量 ===== */
0503     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);    /* 点亮测量指示灯
*/
0504     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);    /* 关闭告警指示灯
*/
0505     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);    /* 关闭蜂鸣器 */
0506
0507     OLED_Clear();
0508     OLED_P8x16Str(0, 0, (unsigned char *)"MEASURING 60s");
0509     sprintf(msg, "L:%d H:%d", bpm_low_limit, bpm_high_limit);
0510     OLED_P8x16Str(0, 2, (unsigned char *)msg);
0511
0512     Send_To_All("Measurement Start, 60s, Fs=50Hz (interrupt-driven)\r\n");
0513     Send_To_All("index,adc,voltage\r\n");
0514
0515     // 非阻塞处理采样队列，确保定时采样中断持续运行。
0516     i = 0;
0517     bpm_live = 0;
0518     start_tick = HAL_GetTick();
0519     last_oled_tick = start_tick;
0520     __disable_irq();
0521     isr_pulse_count = 0;
0522     isr_pulse_high = 0;
0523     adc_queue_head = 0;
0524     adc_queue_tail = 0;
0525     adc_queue_overflow = 0;
0526     adc_ready = 0;
0527     measurement_active = 1;
0528     __enable_irq();
0529
0530     while ((HAL_GetTick() - start_tick) < 60000UL)
0531     {
0532         Key_Process();
0533
0534         /*
0535          * 每次主循环最多处理一个排队采样值，避免长时间停留在队列处理循环中。
0536          * 这样可持续检查 60 秒截止时间和按键事件。
0537          */
0538         if (adc_queue_tail != adc_queue_head)

```

```

0539     {
0540         queued_adc = adc_queue[adc_queue_tail];
0541         adc_queue_tail = (uint16_t)((adc_queue_tail + 1U) % ADC_QUEUE_SIZE);
0542         adc_value = queued_adc;
0543         voltage = adc_value * 3.3f / 4095.0f;
0544         Send_Wave_Sample((uint32_t)i, adc_value);
0545         i++;
0546     }
0547
0548     count_snapshot = isr_pulse_count;
0549     elapsed_ms = HAL_GetTick() - start_tick;
0550     if (elapsed_ms > 0)
0551     {
0552         bpm_live = (int)((count_snapshot * 60000UL) / elapsed_ms);
0553     }
0554
0555     /* 每 5 秒刷新一次实时测量信息。 */
0556     if (HAL_GetTick() - last_oled_tick >= 5000UL)
0557     {
0558         last_oled_tick += 5000UL;
0559         OLED_Clear();
0560         if (key_mode != 0)
0561         {
0562             OLED_Show_Key_Setting();
0563         }
0564         else
0565         {
0566             sprintf(msg, "BPM:%d CNT:%u", bpm_live, count_snapshot);
0567             OLED_P8x16Str(0, 0, (unsigned char *)msg);
0568             elapsed_ms = HAL_GetTick() - start_tick;
0569             if (elapsed_ms > 60000UL) elapsed_ms = 60000UL;
0570             sprintf(msg, "V:%.2fV T:%lus", voltage, elapsed_ms / 1000UL);
0571             OLED_P8x16Str(0, 2, (unsigned char *)msg);
0572             sprintf(msg, "L:%d H:%d", bpm_low_limit, bpm_high_limit);
0573             OLED_P8x16Str(0, 4, (unsigned char *)msg);
0574             OLED_P8x16Str(0, 6, (unsigned char *)"MEASURING");
0575         }
0576     }
0577
0578     /* 不使用长延时，持续处理 50 Hz 采样队列。 */
0579 }
0580
0581 measurement_active = 0;
0582 count_snapshot = isr_pulse_count;

```

```

0583     bpm_final = count_snapshot;    /* 统计时间为 60 秒，因此计数值即为每分钟脉搏数。 */
0584     is_alarm = ((bpm_low_limit > 0 && bpm_final < bpm_low_limit) || (bpm_final >
bpm_high_limit));
0585
0586     Send_To_All("Measurement End\r\n");
0587     sprintf(msg, "Final: Pulse Count=%d, BPM=%d, Time=60s, LOW=%d, HIGH=%d\r\n",
0588             count_snapshot, bpm_final, bpm_low_limit, bpm_high_limit);
0589     Send_To_All(msg);
0590
0591     OLED_Clear();
0592     sprintf(msg, "FINAL BPM:%d", bpm_final);
0593     OLED_P8x16Str(0, 0, (unsigned char *)msg);
0594     sprintf(msg, "COUNT:%u 60s", count_snapshot);
0595     OLED_P8x16Str(0, 2, (unsigned char *)msg);
0596     sprintf(msg, "L:%d H:%d", bpm_low_limit, bpm_high_limit);
0597     OLED_P8x16Str(0, 4, (unsigned char *)msg);
0598
0599     if (is_alarm)
0600     {
0601         OLED_P8x16Str(0, 6, (unsigned char *)"ALARM");
0602         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET); /* 打开蜂鸣器 */
0603         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); /* 点亮告警指示灯 */
0604         Send_To_All("ALARM: BPM out of range!\r\n");
0605     }
0606     else
0607     {
0608         OLED_P8x16Str(0, 6, (unsigned char *)"NORMAL");
0609         HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET);
0610         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
0611     }
0612
0613     HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); /* 关闭测量指示灯 */
0614     HAL_Delay(5000); /* OLED 保留最终测量结果 5 秒。 */
0615
0616     HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); /* 结果显示结束后关
闭蜂鸣器 */
0617     OLED_Clear();
0618     OLED_P8x16Str(0, 0, (unsigned char *)"STANDBY");
0619     OLED_P8x16Str(0, 2, (unsigned char *)"Remove probe");
0620     Wait_Signal_Remove();
0621     /* USER CODE END 3 */
0622 }
0623 }

```

```

0624
0625 /**
0626  * @brief 系统时钟配置
0627  * @retval 无
0628  */
0629 void SystemClock_Config(void)
0630 {
0631     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
0632     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
0633     RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};
0634
0635     /** 按照 RCC_OscInitTypeDef 结构体中的参数初始化 RCC 振荡器。
0636     */
0637     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
0638     RCC_OscInitStruct.HSIState = RCC_HSI_ON;
0639     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
0640     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
0641     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
0642     {
0643         Error_Handler();
0644     }
0645
0646     /** 初始化 CPU、AHB 和 APB 总线时钟。
0647     */
0648     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
0649                                     |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
0650     RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
0651     RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
0652     RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
0653     RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
0654
0655     if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
0656     {
0657         Error_Handler();
0658     }
0659     PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_ADC;
0660     PeriphClkInit.AdcClockSelection = RCC_ADCPCLK2_DIV2;
0661     if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
0662     {
0663         Error_Handler();
0664     }
0665 }
0666
0667 /* USER CODE BEGIN 4 */

```

```

0668
0669 /* USER CODE END 4 */
0670
0671 /**
0672  * @brief  发生错误时执行本函数
0673  * @retval 无
0674  */
0675 void Error_Handler(void)
0676 {
0677     /* USER CODE BEGIN Error_Handler_Debug */
0678     /* 可在此处添加 HAL 错误状态的处理代码。 */
0679     __disable_irq();
0680     while (1)
0681     {
0682     }
0683     /* USER CODE END Error_Handler_Debug */
0684 }
0685
0686 #ifndef  USE_FULL_ASSERT
0687 /**
0688  * @brief  报告 assert_param 错误发生时的源文件名和代码行号
0689  * @param   file: 源文件名指针
0690  * @param   line: 错误所在行号
0691  * @retval 无
0692  */
0693 void assert_failed(uint8_t *file, uint32_t line)
0694 {
0695     /* USER CODE BEGIN 6 */
0696     /* 可在此处添加文件名和行号的错误报告代码。 */
0697     /* USER CODE END 6 */
0698 }
0699 #endif /* USE_FULL_ASSERT */

```